

AI 시대에 맞는 TDD 개발방법론

Agenda

- I. AI 시대에 TDD의 필요성
- II. AI 기반 TDD 실습 (Bowling KATA)
- III. Agentic Engineering은 Over-engineering일까?
- IV. 지속적인 발전을 위한 Agentic Engineering

교육 내용 및 학습 목표

- AI 시대에 TDD가 왜 다시 주목받고 있는지 살펴보고, AI 시대에 어울리는 TDD 방법론에 대해 알아본다.
- 끝으로 Agentic Engineering에 대한 방향성에 대해 살펴본다.

Chapter 01

AI 시대에 TDD의 필요성

SW 품질을 위한 Agentic Engineering

기존 TDD

TDD는 생산성보다 매우 높은 품질을 위한 방법론이다.

장점은 높은 SW 품질 확보이며,

단점으로는 테스트 코드 작성과 리팩토링에 드는 시간과 비용 때문에 느리고 비효율적이라는 인식이 존재했다.

AI 시대가 되면서 TDD가 더 편해졌다.

AI의 생산성의 도움을 받으면, 빠른 테스트코드 작성과 리팩토링이 가능하다.

이제 TDD는 더 이상 답답하고 부담스러운 방식이 아니다.

AI의 생산성이 결합되면서, 개발자는 높은 품질을 유지하면서도 실질적으로 활용 가능한 수준의 속도와 효율을 확보할 수 있게 되었다.

즉, AI 시대의 TDD는

“느리지만 좋은 방법”이 아니라,

“빠르면서도 품질을 보장하는 현실적인 전략”으로 재해석되고 있다.

1. 지시의 모호성 축소

- Text 기반으로 요구사항을 전달하면, 무엇을 만들어야 하는지 구체적이지 않을 수 있다.
- 반면 TDD는 Unit Test를 먼저 작성함으로써 AI에게 요구사항을 Code 기반으로 명확히 지시할 수 있다.
- 사람도 Text 지시에 대해 모호성을 겪지만, AI는 이를 더 크게 확대해석한다.
- 테스트는 “무엇을 만들어야 하는 지”를 명확하게 고정해준다.
- Text가 아닌 코드 기반 명세이기 때문에 해석 오차가 줄어든다.
- 즉, Text 지시가 아닌 Code로 지시하면, 잘못된 방향으로 구현될 확률을 낮춘다.

2. 안전장치 역할 (가장 중요)

- AI가 생성하는 코드는 빠르지만, 항상 신뢰할 수 있는 것은 아니다.
- TDD는 다음과 같은 구조를 만든다.
 - 먼저 테스트를 정의한다 (기대 결과 고정)
 - 그 다음 AI에게 구현을 맡긴다.
 - 여기서 Unit Test는 가드레일(안전장치) 역할을 한다.
 - 잘못된 구현은 Test Fail로 즉시 드러난다
 - 예상 범위를 벗어나는 코드가 만들어지기 어렵다.

3. 검토(리뷰)가 쉬워진다

- Test Code를 고민하면서 만들었다면, Green Code가 만들어지기 전 어떤 코드가 만들어질지 예측이 된다.
- 예측 범위 내에서 코드가 만들어지기에 AI가 만든 코드의 리뷰 속도가 굉장히 빠르다.

Chapter 03

AI 기반 TDD 실습 (Bowling KATA)

SW 품질을 위한 Agentic Engineering

AI 시대에 맞는 TDD

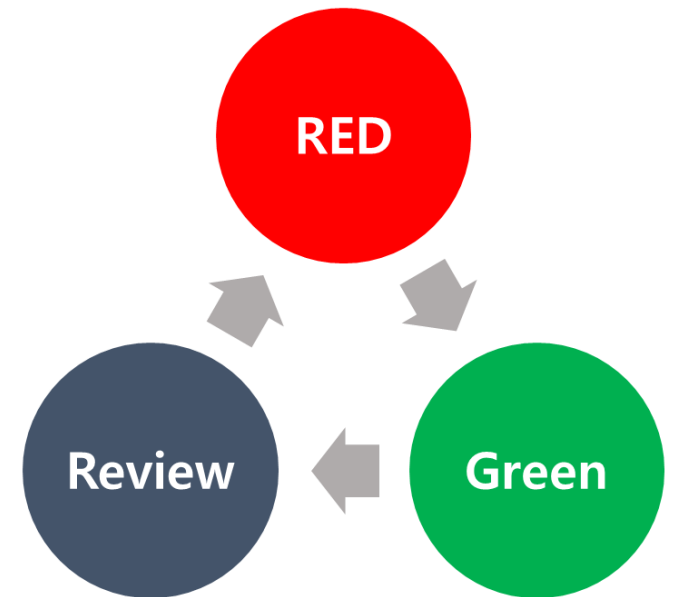
AI 시대에는 기존 방식에 더해, AI를 효과적으로 활용할 수 있는 Customize TDD가 필요하다. 이번 실습에서는 두 가지 방법의 AI + TDD를 단계적으로 경험해본다.

1. Original TDD 방식에 AI를 활용

- 기존 TDD 원칙을 유지한 상태에서 AI를 보조 도구로 활용한다.
- Bowling KATA를 통해 AI + TDD 사이클을 경험한다. (Superpower Skills 사용)

2. Agentic Engineering을 적용한 TDD 방식

- AE 원칙을 적용한 Red-Green-Review Cycle을 이해하고 경험한다.
- Bowling KATA를 Agentic Engineering 방식으로 재구성하여 실습한다.



Bowling kata 소개

Bowling 점수 산출 코드 작성 TDD

볼링 kata는 클린코드 저자로 유명한 Robert C. Martin 이 만든 TDD 연습용 kata이다.

실제 볼링 게임을 만드는 것이 아니라,

볼링 점수를 계산하는 로직과 설계를 TDD Cycle 로 연습해본다.

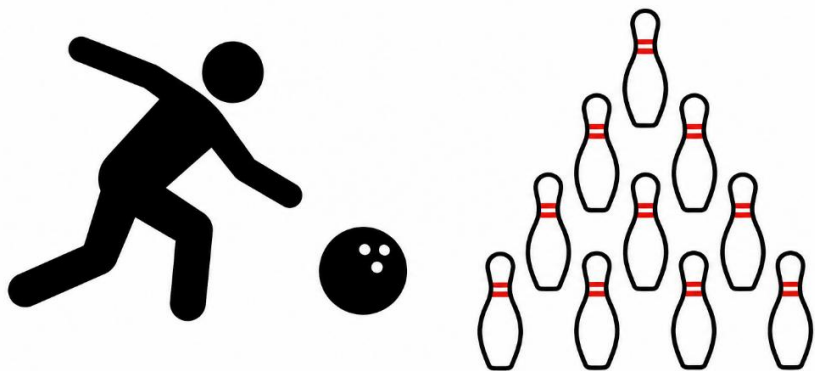
→ Coding Agent를 사용해서 TDD Cycle 을 돌려본다

Bowling 규칙

볼링 경기는 10 개의 볼링 핀을 넘어뜨리는 게임이다.

총 10 라운드가 진행되며, 각 라운드별로 최대 2번의 볼링공을 굴릴 기회가 주어진다.

다음 라운드에 볼링 핀은 다시 10개로 초기화된다.



각 라운드별로 **최대** 2번
굴릴 기회가 있다.

10 라운드는 예외로
최대 3번까지 굴릴 기회가 주어진다

1	2	3	4	5	6	7	8	9	10

볼링 점수판

기본 점수 규칙

한 라운드에서 얻을 수 있는 볼링의 점수는 공을 굴러 넘어뜨린 핀의 개수로 결정된다.
단, Spare 와 Strike 는 보너스 점수가 있다.

Round	
3	2

첫번째에는 3개, 두번째에는 2개를 쓰러뜨렸다.
→ 해당 라운드에서 얻은 점수 = $3 + 2$

Round	
2	/

첫번째에는 2개, 두번째에는 8개를 쓰러뜨려
한 라운드에 10개 모두 쓰러뜨렸다
→ 이를 Spare 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $2 + 8 + \text{보너스}$

Round	
X	

첫번째 시도에 10개를 모두 쓰러뜨렸다.
→ 이를 Strike 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $10 + \text{보너스}$

Spare 와 Strike 의 보너스 점수 규칙

한 라운드에 10개의 볼링 핀을 넘어뜨리면 Spare 또는 Strike 로 판정된다.

Spare 인 경우 **다음 1 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 하고,

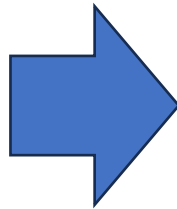
Strike 인 경우 **다음 2 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 합산한다.

1Round		2Round	
2	/	3	5

1 Round 에서 얻는 점수
 $= 2 + 8 + 3$

1Round		2Round	
X		3	5

1 Round 에서 얻는 점수
 $= 10 + 3 + 5$



1Round		2Round		3Round	
X		X		X	

Q. 1,2,3 Round 에서 모두 Strike 인 경우
1 Round 에서 얻는 점수는?

마지막 10 Round 보너스

마지막 10 Round 는 Spare, Strike 인 경우 보너스 점수용 기회 포함 **3번 공을 굴릴 수 있다.**
단, 보너스 점수용 기회는 Strike이나 Spare이더라도 추가 기회는 없다.

10Round		
2	/	보너스

Spare 이므로 보너스 점수용 기회를 **1번** 준다.

10Round		
X	보너스	보너스

Strike 이므로 보너스 점수용 기회를 **2번** 준다.

Bowling 점수 예시

다음 볼링 점수판 예시를 보고 점수 규칙을 잘 이해했는지 확인해본다.

12	1	2	3	4	5	6	7	8	9	10	TTL
David	8 /	9 -	9 /	X	7 1	X	6 3	X	6 3	X 8 -	148
	19	28	48	66	74	93	102	121	130	148	
Smith	X	9 /	7 -	9 /	X	X	8 /	X	9 /	9 / ⑦	188
	20	37	44	64	92	112	132	152	171	188	
Green	4 -	5 -	8 -	X	1 8	1 -	6 /	8 -	9 -	7 1	89
	4	9	17	36	45	46	64	72	81	89	
Lazzari	9 /	5 4	3 /	7 /	X	6 -	6 -	⑧ 1	5 4		107
	15	24	41	61	77	83	89	98	107		
Hdcp	2/3	58	98	150	227	288	334	387	443	489	532

1 번째 쓰러뜨린 개수	2 번째 쓰러뜨린 개수
1 ~ n 라운드 누적점수	

cpp 프로젝트 생성

cpp 프로젝트 생성

Project Name : bowling-tdd

새 프로젝트 구성

빈 프로젝트

C++

Windows

콘솔

프로젝트 이름(I)
bowling-tdd

위치(L)
C:\reviewer\workspace

솔루션(S)
새 솔루션 만들기

솔루션 이름(M) ⓘ
bowling-tdd

☐ 솔루션 및 프로젝트를 같은 디렉터리에 배치(D)

"C:\reviewer\workspace\bowling-tdd\bowling-tdd*"에 프로젝트이(가) 만들어집니다.

2026년 2월 기능 업데이트

뒤로(B)

만들기(C)

프로젝트 세팅

agent 에게 기본 실습 세팅 요청하고, 정상적으로 동작되는지 체크까지 진행한다.

현재 프로젝트는 visual studio 로 제작되었다.
hello world 출력 만들고, msbuild 로 실행해줘



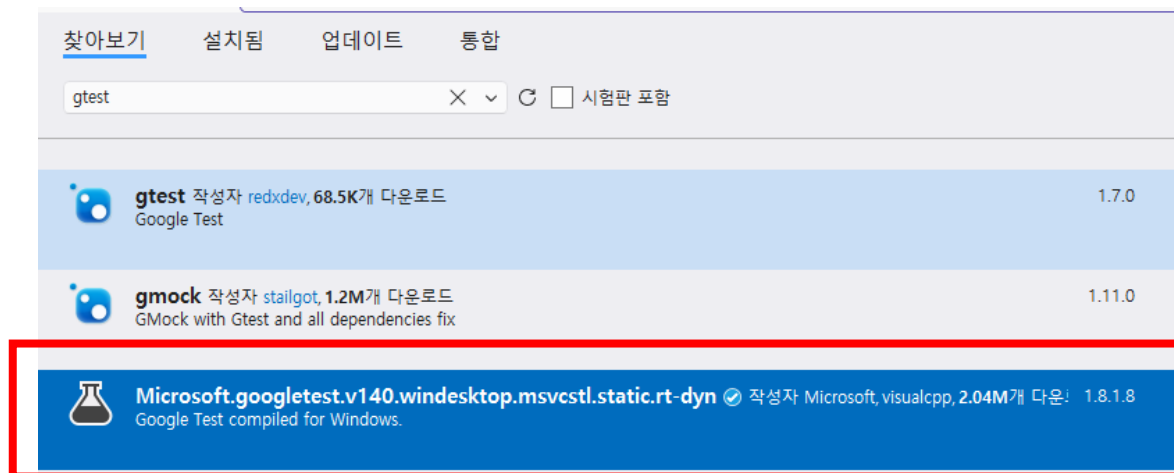
</> Edit automatically



gtest 으로 main.cpp 에 간단한 assertion 함수
작성해주고, msbuild 로 test 실행해줘. 현재
프로젝트에 NuGet으로 gtest 설치했어.



</> Edit automatically



visual studio 에서
nuget 을 사용해 미리
설치해놓는다.
(호환성 문제로 Microsoft.googletest.v140 설치)

CLAUDE.md 생성

/init skill을 사용하여 AGENTS.md 문서 생성 후

Bowling kata 내용 기입하기

기입할 내용: <https://gist.github.com/jeonghwan-seo/662e65f320fe0f0f3ce654ed464e89a8>



/init 빌드 및 실행방법 정리해줘.

</> Edit automatically 



/init @bowling-kata.md 내용 정리해서 기입

</> Edit automatically 



CLAUDE.md 에 test는 gtest 로 작성함을 명시해줘

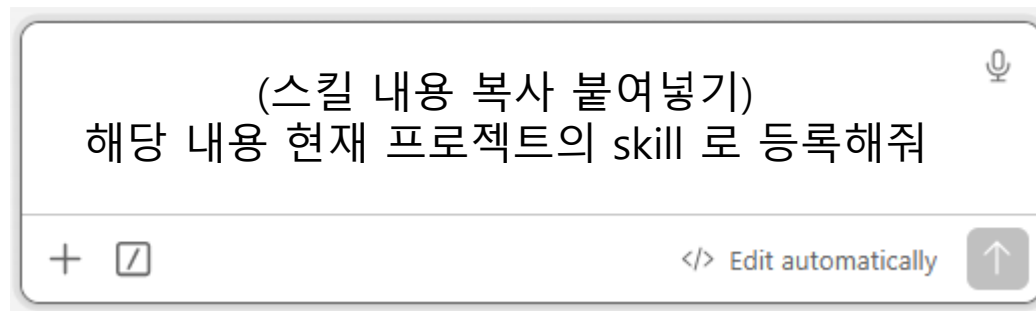
 

</> Edit automatically 

Skill 세팅하기

이번 실습은 superpowers 의 TDD Skill 을 가져와서 사용해본다.

- ✓ superpowers TDD skill : <https://github.com/obra/superpowers/tree/main/skills/test-driven-development>
- ✓ 언어별 번역본 : <https://gist.github.com/jeonghwan-seo/fd1a2cd6a2d3cc00b8a3d5a68a21cc2f>

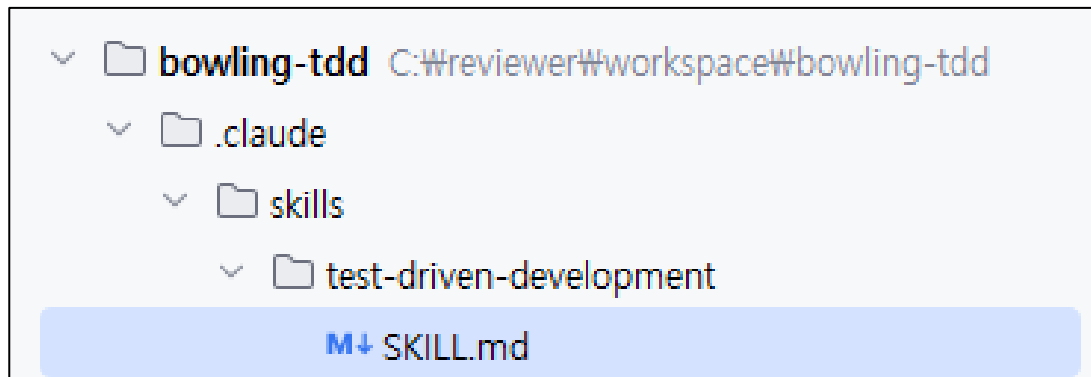
A screenshot of a skill creator interface. It features a text input area with the Korean text "(스킬 내용 복사 붙여넣기)" and "해당 내용 현재 프로젝트의 skill 로 등록해줘". To the right of the text is a microphone icon. Below the text area is a toolbar with a plus icon, a link icon, a code editor icon labeled "</> Edit automatically", and an upward arrow icon.

skill creator 가 실행되어서 새로 만들어주거나,
기존내용을 그대로 사용하게 된다.

```
● skill-creator를 사용하여 새로운 스킬을 등록하겠습니다.  
● Skill(skill-creator)  
  ↳ Successfully loaded skill
```

Skill 수동으로 세팅하기

스킬이 잘 등록 안될경우 .claude/skills/ 에 다음과 같이 등록



TDD Cycle 검토

기본적으로 Skill 에 명시된 대로 TDD 가 진행되는지 확인한다.

RED 단계에서 검토해야 할 부분 :

- ✓ 테스트는 구현에 대한 계획과 목표점이 된다.
- ✓ 테스트 작성을 제일 먼저 진행했는지
- ✓ 요구사항에 맞는 테스트 코드를 만들고 테스트 수행을 했는지

GREEN 단계에서 검토해야 할 부분:

- ✓ RED 단계의 테스트가 요구를 벗어난 불필요한 코드를 생성하진 않았는지 검토
- ✓ 테스트를 수행하고 PASS 되는지 확인

스킬 명시적 사용

스킬은 따로 언급하지 않아도 자연스럽게 발동될 수 있다.

하지만, 확실하게 사용하고 싶으면 명시적으로 스킬을 언급하도록 한다.

Bowling kata 구현해줘

+  </> Edit automatically 

```
> 볼링 카타 구현해줘
```

- 볼링 카타를 TDD 원칙대로 구현하겠습니다. `.claude/skills/test-driven-development` 가이드에 따라 RED → GREEN을 사이클별로 직접 보면서 진행합니다.

TDD 로 Bowling kata 구현해줘
test-driven-development 스킬 따라서 Bowling kata 구현해줘.
/test-driven-development Bowling kata 요구사항 파악하고
구현해줘

+  </> Edit automatically 

```
> tdd 로 볼링 카타 구현해줘
```

- Skill(test-driven-development)
 - L Successfully loaded skill



명시적인 사용

요구사항 파악 및 TDD 개발 진입

요구사항이 정리 되어있는 문서를 잘 읽고 이해했는지 파악한다.

- 먼저 현재 프로젝트 구조를 파악하겠습니다.

Searched for 3 patterns, read 2 files (ctrl+o to expand)

- 프로젝트 구조를 파악했습니다. Java 소스가 없는 깨끗한 스캐폴드입니다. CLAUDE.md의 볼링 요구사항을 바탕으로 TDD로 구현하겠습니다.

요구사항 정리:

- Game 클래스: `void roll(int pins), int score()`
- 스페어: 다음 1구 보너스
- 스트라이크: 다음 2구 보너스
- 10번 프레임: 스트라이크/스페어 시 추가 투구

TDD 사이클: 실패하는 테스트 → GREEN → 다음 테스트 순서로 진행합니다.

RED 1 – 거터 게임 (전부 0)

가장 단순한 기능에 대한 테스트를 추가하고, 테스트 실패를 확인한다.

- ✓ 스트라이크, 스페어 등의 여러 요구사항을 배제한 테스트부터 시작
- ✓ 코드 뼈대 형성 가능

RED 1 – 거터 게임 (전부 0)

먼저 테스트 파일을 작성합니다.

```
#include <gtest/gtest.h>
#include "Game.h"

TEST(GameTest, GutterGameScoresZero) {
    Game game;
    for (int i = 0; i < 20; ++i) {
        game.Roll(0);
    }
    EXPECT_EQ(game.Score(), 0);
}

int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

● 빌드 성공! 이제 테스트 실행 (RED phase 확인):

```
● PowerShell(.\\bowling-tdd-kata\\x64\\Debug\\bowling-tdd-kata.exe)
  L [=====] Running 1 test from 1 test case.
  [-----] Global test environment set-up.
  [-----] 1 test from GameTest
  ... +7 lines (ctrl+o to expand)
```

● 문제 발견: 테스트가 통과했지만, TDD 스킬에 따르면 이는 기존 동작을 테스트하고 있다는 뜻입니다. 더 의미 있는 테스트를 추가하겠습니다.

RED – 전부 1개의 볼링핀

점수 누적에 대한 테스트 추가

좋습니다! RED phase 확인 - AllOnesScores20 테스트 실패. 이제 GREEN phase - 최소 구현:

```
TEST(GameTest, AllOnesScores20) {  
    ...  
    Game game;  
    for (int i = 0; i < 20; ++i) {  
        game.Roll(1);  
    }  
    EXPECT_EQ(game.Score(), 20);  
}
```

RED 테스트 추가

GREEN 2 – 전부 1개의 볼링핀

최소 구현으로 통과 확인

```
• Update(bowling-tdd-kata\Game.cpp)
  L Added 5 lines, removed 1 line
    5 }
    6
    7 int Game::Score() {
    8 - return 0;
    8 + int score = 0;
    9 + for (int i = 0; i < current_roll; ++i) {
   10 +     score += rolls[i];
   11 + }
   12 + return score;
   13 }
```

GREEN 최소구현으로 통과

```
• PowerShell(cd 'C:\reviewer\workspa
  Studio\18\Community\MSBu
  L 오류 0개

  경과 시간: 00:00:03.60
  ... +13 lines (ctrl+o to expand)

• 완벽합니다! 모두 통과. 이제 스페어
```

RED 3 – 스페어 테스트 추가

스페어 점수에 대한 테스트 추가

- 완벽합니다! 모두 통과. 이제 스페어(Spare) 처리 테스트를 RED로 작성:

```
● Update(bowling-tdd-kata\main.cpp)
  L Added 11 lines
  17     EXPECT_EQ(game.Score(), 20);
  18 }
  19
  20 +TEST(GameTest, OneSpareAndThreeThenZeros) {
  21 +     Game game;
  22 +     game.Roll(5);
  23 +     game.Roll(5); // Spare
  24 +     game.Roll(3);
  25 +     for (int i = 0; i < 17; ++i) {
  26 +         game.Roll(0);
  27 +     }
  28 +     EXPECT_EQ(game.Score(), 16); // 10 + 3 + 3 = 16
  29 +}
  30 +
```

spare의 보너스 규칙에 대한
테스트를 생성했는지 검토

영과 자인: 00.00.00.72

```
[ RUN      ] GameTest.OneSpareAndThreeThenZeros
C:\reviewer\workspace\bowling-tdd-kata\bowling-
game.Score()
Which is: 13
16
... +10 lines (ctrl+o to expand)
```

- RED 확인 (13 vs 16).

스페어에 대한 기능 추가 전
→ FAIL 확인

GREEN 3 – 스페어 보너스 구현

Spare 점수에 대한 구현 추가

● RED 확인 (13 vs 16). GREEN 구현 - 스페어 보너스 처리:

● Update(C:\reviewer\workspace\bowling-tdd-kata\bowling-tdd-kata\Game.cpp)

└ Added 10 lines, removed 2 lines

```
6
7  int Game::Score() {
8      int score = 0;
9 -   for (int i = 0; i < current_roll; ++i) {
10 -       score += rolls[i];
11 +   int roll_index = 0;
12 +   for (int frame = 0; frame < 10; ++frame) {
13 +       if (rolls[roll_index] + rolls[roll_index + 1] == 10) { // Spare
14 +           score += 10 + rolls[roll_index + 2];
15 +           roll_index += 2;
16 +       } else {
17 +           score += rolls[roll_index] + rolls[roll_index + 1];
18 +           roll_index += 2;
19 +       }
20 +   }
21 +   return score;
22 }
```

```
[=====] Running 3 tests from 1 test case.
[-----] Global test environment set-up.
[-----] 3 tests from GameTest
[ RUN      ] GameTest.GutterGameScoresZero
[          OK ] GameTest.GutterGameScoresZero (0 ms)
[ RUN      ] GameTest.AllOnesScores20
[          OK ] GameTest.AllOnesScores20 (0 ms)
[ RUN      ] GameTest.OneSpareAndThreeThenZeros
[          OK ] GameTest.OneSpareAndThreeThenZeros (0 ms)
[-----] 3 tests from GameTest (1 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test case ran. (1 ms total)
[ PASSED   ] 3 tests.
```

RED 단계의 Spare 점수에 대한 테스트 실행
→ PASS 가 되어야 함

RED 4 – 스트라이크 테스트 추가

스트라이크 점수에 대한 테스트 추가

- 완벽! 모두 통과. 이제 스트라이크(Strike) 처리 테스트를 RED로 작성:

- Update(bowling-tdd-kata\main.cpp)

└ Added 11 lines

```
28     EXPECT_EQ(game.Score(), 16); // 10 + 3 + 3 = 16
29 }
30
31 +TEST(GameTest, OneStrikeAndThreeFourThenZeros) {
32 +     Game game;
33 +     game.Roll(10); // Strike
34 +     game.Roll(3);
35 +     game.Roll(4);
36 +     for (int i = 0; i < 16; ++i) {
37 +         game.Roll(0);
38 +     }
39 +     EXPECT_EQ(game.Score(), 24); // 10 + 3 + 4 + 3 + 4 = 24
40 +}
41 +
42 int main(int argc, char** argv) {
43     ::testing::InitGoogleTest(&argc, argv);
44     return RUN_ALL_TESTS();
}
```

스트라이크 보너스 점수에 대한
테스트를 생성했는지 검토

```
[ OK ] GameTest.GutterGameScoresZero (0 ms)
[ RUN ] GameTest.AllOnesScores20
[ OK ] GameTest.AllOnesScores20 (0 ms)
[ RUN ] GameTest.OneSpareAndThreeThenZeros
[ OK ] GameTest.OneSpareAndThreeThenZeros (0 ms)
[ RUN ] GameTest.OneStrikeAndThreeFourThenZeros
C:\reviewer\workspace\bowling-tdd-kata\bowling-tdd-kata\main.cpp(39):
game.Score()
Which is: 17
24
[ FAILED ] GameTest.OneStrikeAndThreeFourThenZeros (0 ms)
[-----] 4 tests from GameTest (0 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test case ran. (1 ms total)
[ PASSED ] 3 tests.
[ FAILED ] 1 test, listed below:
[ FAILED ] GameTest.OneStrikeAndThreeFourThenZeros

1 FAILED TEST
```

- 예상대로 실패합니다.

스트라이크에 대한 기능 추가 전
→ FAIL 확인

GREEN 4 – 스트라이크 보너스 구현

스트라이크 기능에 대한 테스트가 통과할 수 있도록 구현

- RED 확인 (17 vs 24). GREEN 구현 - 스트라이크 처리:

- Update(bowling-tdd-kata\Game.cpp)

└ Added 4 lines, removed 1 line

```
9     int roll_index = 0;
10
11     for (int frame = 0; frame < 10; ++frame) {
12 -     if (rolls[roll_index] + rolls[roll_index + 1] == 10) { // Spare
12 +     if (rolls[roll_index] == 10) { // Strike
13 +         score += 10 + rolls[roll_index + 1] + rolls[roll_index + 2];
14 +         roll_index += 1;
15 +     } else if (rolls[roll_index] + rolls[roll_index + 1] == 10) { // Spare
16         score += 10 + rolls[roll_index + 2];
17         roll_index += 2;
18     } else {
```

- Update(bowling-tdd-kata\main.cpp)

[OK] GameTest.OneStrikeAndThreeFourThenZeros (0 ms)

[-----] 4 tests from GameTest (0 ms total)

[-----] Global test environment tear-down

[=====] 4 tests from 1 test case ran. (0 ms total)

[PASSED] 4 tests.

RED/GREEN 5 – 퍼펙트 게임 테스트(모두 스트라이크)

마지막 10라운드에서의 규칙도 적용되도록 테스트와 구현을 한다.

```
● Update(C:\reviewer\workspace\bowling-tdd-kata\bowling-tdd-kata\main.cpp)
  L Added 8 lines
  39     EXPECT_EQ(game.Score(), 24); // 10 + 3 + 4 + 3 + 4 = 24
  40 }
  41
  42 +TEST(GameTest, PerfectGameScores300) {
  43 +     Game game;
  44 +     for (int i = 0; i < 12; ++i) {
  45 +         game.Roll(10); // 12 strikes: 10 frames + 2 bonus rolls for 10th frame
  46 +     }
  47 +     EXPECT_EQ(game.Score(), 300);
  48 + }
  49 +
  50 int main(int argc, char** argv) {
  51     ::testing::InitGoogleTest(&argc, argv);
  52     return RUN_ALL_TESTS();
  }
```

```
[=====] Running 5 tests from 1 test case.
[-----] Global test environment set-up.
[-----] 5 tests from GameTest
[ RUN    ] GameTest.GutterGameScoresZero
[ OK     ] GameTest.GutterGameScoresZero (0 ms)
[ RUN    ] GameTest.AllOnesScores20
[ OK     ] GameTest.AllOnesScores20 (0 ms)
[ RUN    ] GameTest.OneSpareAndThreeThenZeros
[ OK     ] GameTest.OneSpareAndThreeThenZeros (0 ms)
[ RUN    ] GameTest.OneStrikeAndThreeFourThenZeros
[ OK     ] GameTest.OneStrikeAndThreeFourThenZeros (0 ms)
[ RUN    ] GameTest.PerfectGameScores300
[ OK     ] GameTest.PerfectGameScores300 (0 ms)
[-----] 5 tests from GameTest (0 ms total)

[-----] Global test environment tear-down
[=====] 5 tests from 1 test case ran. (0 ms total)
[ PASSED ] 5 tests.
```

새로운 테스트, but 기존 구현으로 통과
→ 테스트가 이전 구현으로 우연히 통과인지 체크
→ 이전 구현이 너무 과도하지 않았는지 체크

- 어라, 이미 통과했습니다! 현재 코드가 10번 프레임을 올바르게 처리하고 있다는 뜻입니다. 더 많은 엡지 케이스를 테스트하겠습니다.

RED/GREEN 6 – 10번 프레임에서 스페어 처리 테스트를 추가

엣지 케이스를 추가로 검증하는 테스트 케이스 추가

```
● Update(bowling-tdd-kata\main.cpp)
  L Added 8 lines
    47     EXPECT_EQ(game.Score(), 300);
    48   }
    49
    50 +TEST(GameTest, AllSparesWith5PinsScores150) {
    51 +   Game game;
    52 +   for (int i = 0; i < 21; ++i) {
    53 +     game.Roll(5); // 21 rolls: 10 frames of spare + 1 bonus
    54 +   }
    55 +   EXPECT_EQ(game.Score(), 150);
    56 +}
    57 +
    58 int main(int argc, char** argv) {
    59   ::testing::InitGoogleTest(&argc, argv);
    60   return RUN_ALL_TESTS();
  }
```

```
[=====] Running 6 tests from 1 test case.
[-----] Global test environment set-up.
[-----] 6 tests from GameTest
[ RUN    ] GameTest.GutterGameScoresZero
[ OK     ] GameTest.GutterGameScoresZero (0 ms)
[ RUN    ] GameTest.AllOnesScores20
[ OK     ] GameTest.AllOnesScores20 (0 ms)
[ RUN    ] GameTest.OneSpareAndThreeThenZeros
[ OK     ] GameTest.OneSpareAndThreeThenZeros (0 ms)
[ RUN    ] GameTest.OneStrikeAndThreeFourThenZeros
[ OK     ] GameTest.OneStrikeAndThreeFourThenZeros (0 ms)
[ RUN    ] GameTest.PerfectGameScores300
[ OK     ] GameTest.PerfectGameScores300 (0 ms)
[ RUN    ] GameTest.AllSparesWith5PinsScores150
[ OK     ] GameTest.AllSparesWith5PinsScores150 (0 ms)
[-----] 6 tests from GameTest (0 ms total)

[-----] Global test environment tear-down
[=====] 6 tests from 1 test case ran. (1 ms total)
[ PASSED ] 6 tests.
```

Refactor 단계

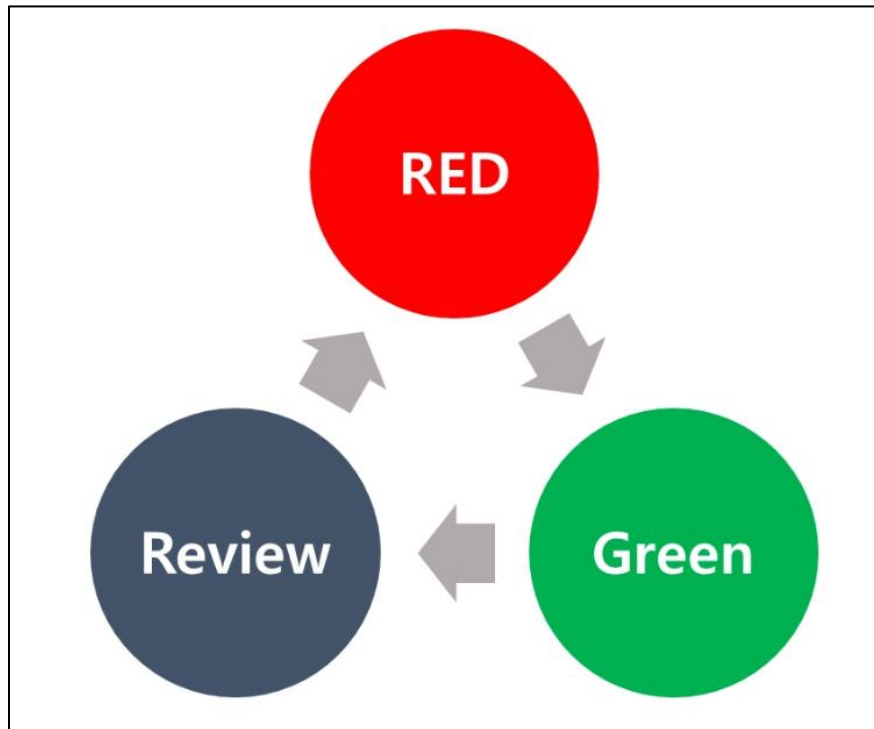
마지막에 리팩터링 페이지로 진입한다

- 완벽합니다! 모든 7개 테스트 통과. 이제 REFACTOR phase - 코드 정리:

```
● Update(bowling-tdd-kata\Game.cpp)
  L Added 12 lines, removed 4 lines
    4     rolls[current_roll++] = pins;
    5 }
    6
    7 -int Game::Score() {
    7 +bool Game::IsStrike(int roll_index) const {
    8 +     return rolls[roll_index] == 10;
    9 +}
   10 +
   11 +bool Game::IsSpare(int roll_index) const {
   12 +     return rolls[roll_index] + rolls[roll_index + 1] == 10;
   13 +}
   14 +
   15 +int Game::Score() const {
   16     int score = 0;
   17     int roll_index = 0;
   18
   19     for (int frame = 0; frame < 10; ++frame) {
   12 -         if (rolls[roll_index] == 10) { // Strike
   20 +         if (IsStrike(roll_index)) {
   21             score += 10 + rolls[roll_index + 1] + rolls[roll_index + 2];
   14 -             roll_index += 1;
   15 -         } else if (rolls[roll_index] + rolls[roll_index + 1] == 10) { // Spare
   22 +             ++roll_index;
   23 +         } else if (IsSpare(roll_index)) {
   24             score += 10 + rolls[roll_index + 2];
   25             roll_index += 2;
   26         } else {
```

Agentic Engineering TDD 리뷰

human-in-the-loop 를 강조하여 Agent 를 사용한 TDD 를 진행해본다.



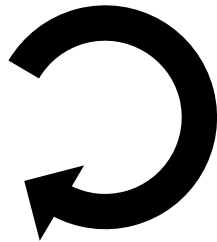
1. RED
 - 목표 정확히 설정
 - Plan.md 생성 요청 후 검토
2. Green
 - RED 에서 생성된 goal 을 달성하도록 구현
 - 테스트를 실제로 통과하는지 체크
3. Review
 - Green 의 코드를 리뷰
 - Plan 외의 사항이 구현되었는지
 - 리팩토링이 필요하다면 요청하기

프로젝트 수행시 커밋할 부분

agentic engineering 수행을 체크하기 위한 용도로,
각 단계별 필요한 커밋들을 생성한다.



RED 단계에서 생성된
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



프로젝트 제출방법 - 프로젝트

GitHub 에 Remote Repository를 생성

Repository 이름

- repo 이름 : **bowling-사번**
- Description : 소속 & 이름
- 접근 권한 : Public

Create a new repository

A repository contains all project files, including the revision history.

Owner

Repository name *

 reviewer-01 ▾

 /

bowling-사번

Great repository names are short and memorable. Need inspiration? How about [redesigned-journey?](#)

Description (optional)

소속 & 이름

☒ Public

 Any logged in user can see this repository. You choose who can commit.

Skip this step if you're importing an existing repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer.

Add .gitignore: **None** ▾

Create repository

Skill Update하기

human-in-the-loop 를 강조하여 TDD 를 진행하기 위해,
필요한 내용들을 적은 후 기존 tdd 스킬을 업데이트한다.

```
> @skill-update-plan.md 를 파악해서 local project 의 /test-driven-development skill 을 업데이트  
해 줘  
L Read skill-update-plan.md (32 lines)
```

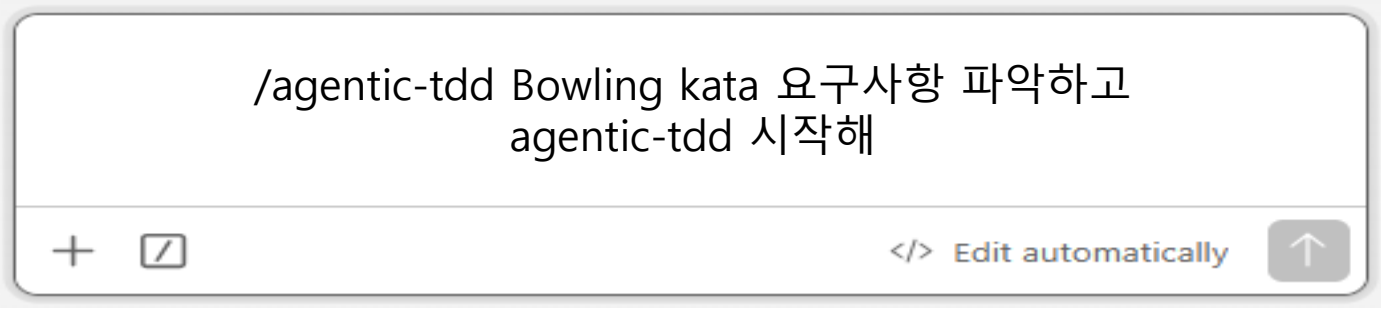
agentic engineering 을 TDD 에서 실천할 수 있도록
문서를 작성해놓는다.

새롭게 작성된 Skill 가동

update 된 skill 을 사용해서 Agentic Engineering 에 맞는 TDD 를 진행한다.

- ✓ 한 사이클을 수행하면서 Agentic Engineering 을 잘 지키는지 Skill 검토가 필요하다.
- ✓ Agentic Engineering 을 잘 수행하지 않으면 Skill을 수정한다.

```
/agentic-tdd Bowling kata 요구사항 파악하고  
agentic-tdd 시작해
```



RED 1 – Plan.md

Agent 가 작성한 Plan.md 를 검토한다.

```
# RED 단계 계획 - 1단계: Game 클래스 및 기본 점수 계산

## 목표
Game 클래스의 기본 구조를 만들고, 간단한 게임(핀을 쓰러뜨리지 않은 경우)의 점수를 정확히 계산하는 기능을 구현합니다.

## 요구사항 분석

### 첫 번째 테스트: `GameTest_AllGutters_ScoresZero`
- 게임의 모든 20번 투구가 0핀(거터)인 경우
- 예상 점수: 0점
- 이 테스트는 Game 클래스의 기본 틀과 roll(), score() 메서드의 기본 구조를 검증합니다

## 구현 계획

### 헤더 파일: `Game.h`
```cpp
#pragma once

class Game {
public:
 void roll(int pins);
 int score();

private:
 int rolls[21] = {0}; // 최대 21번 투구 (10번째 프레임 최대 3번)
 int current_roll = 0; // 현재 투구 인덱스
};
```

### 구현 파일: `Game.cpp`
```cpp
#include "Game.h"

void Game::roll(int pins) {
 rolls[current_roll++] = pins;
}
```

-  RED 단계 완료 – Plan.md를 작성했습니다.

RED-1-plan.md에서 첫 번째 기능의 계획을 수립했습니다:

- 테스트: 모든 투구가 거터(0핀)인 게임의 점수 = 0
- 구현: Game 클래스, roll(), score() 메서드의 기본 구조
- 파일: Game.h, Game.cpp, main.cpp 생성

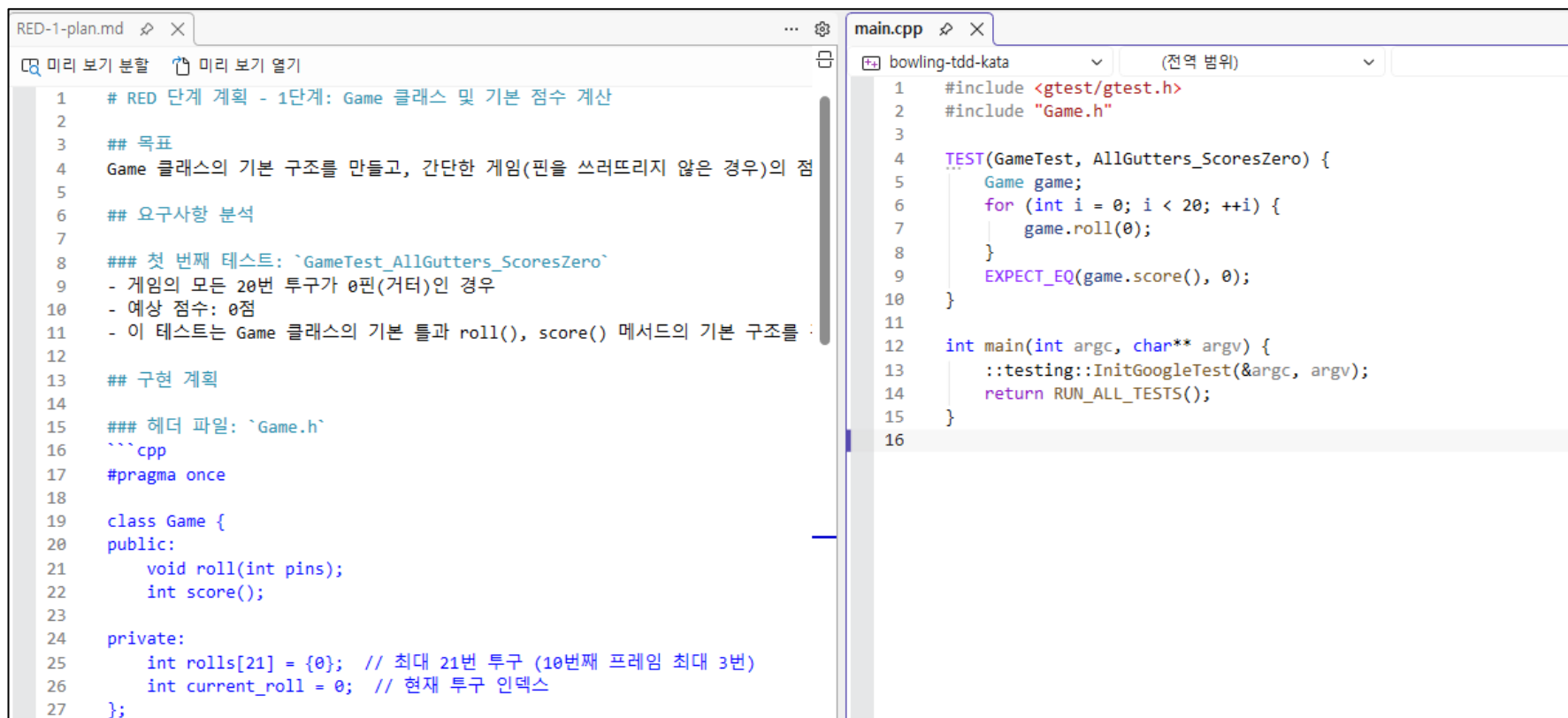
검토를 요청합니다. 계획이 적절한지 검토해주세요. 승인이 나면 다음 단계로 진행하겠습니다.

# RED 1 – Test Code 변환 후 검토

Plan.md 가 마음에 들었다면 다음 step 을 진행시킨다.

Plan.md → Test Code (plan) 으로 변환작업을 수행

사람은 Plan 에 맞게 Test Code 를 만들었는지 확인한다.



```
RED-1-plan.md
1 # RED 단계 계획 - 1단계: Game 클래스 및 기본 점수 계산
2
3 ## 목표
4 Game 클래스의 기본 구조를 만들고, 간단한 게임(핀을 쓰러뜨리지 않은 경우)의 점
5
6 ## 요구사항 분석
7
8 ### 첫 번째 테스트: `GameTest_AllGutters_ScoresZero`
9 - 게임의 모든 20번 투구가 0핀(거터)인 경우
10 - 예상 점수: 0점
11 - 이 테스트는 Game 클래스의 기본 틀과 roll(), score() 메서드의 기본 구조를 :
12
13 ## 구현 계획
14
15 ### 헤더 파일: `Game.h`
16 ```cpp
17 #pragma once
18
19 class Game {
20 public:
21 void roll(int pins);
22 int score();
23
24 private:
25 int rolls[21] = {0}; // 최대 21번 투구 (10번째 프레임 최대 3번)
26 int current_roll = 0; // 현재 투구 인덱스
27 };

main.cpp
1 #include <gtest/gtest.h>
2 #include "Game.h"
3
4 TEST(GameTest, AllGutters_ScoresZero) {
5 Game game;
6 for (int i = 0; i < 20; ++i) {
7 game.roll(0);
8 }
9 EXPECT_EQ(game.score(), 0);
10 }
11
12 int main(int argc, char** argv) {
13 ::testing::InitGoogleTest(&argc, argv);
14 return RUN_ALL_TESTS();
15 }
16
```

## GREEN 1 – 테스트를 통과시킬 최소 구현

Test Code 를 기반으로 빠르게 코드를 생성 후, Test Code 를 돌려본다.

```
#include "Game.h"

void Game::roll(int pins) {
}

int Game::score() {
 return 0;
}
```

# REVIEW 1 – 생성된 코드 리뷰

코드를 검토한다.

Plan(Test Code) 에 맞도록 구현되었는가

- 지나치게 일반화 되거나, 불필요한 부분을 추가하지 않았는 지
- 리팩토링이 필요한 지

---  
❌ CODE REVIEW - 코드를 검토해 주세요.

검토 후 다음 사이클(두 번째 테스트)로 넘어가겠습니다.

```
#include "Game.h"

void Game::roll(int pins) {
}

int Game::score() {
 return 0;
}
```



# RED 2 – Plan.md

다음 사이클에서도 Plan.md 를 받는다.

```
RED-1-plan.md ✨ X ...
미리 보기 분할 미리 보기 열기

1 # RED 단계 계획 - 1단계: Game 클래스 및 기본 점수 계산
2
3 ## 목표
4 Game 클래스의 기본 구조를 만들고, 간단한 게임(핀을 쓰러뜨리지 않은 경우)의 점수를 정확히 계산하는 기능을 구현합니다.
5
6 ## 요구사항 분석
7
8 ### 첫 번째 테스트: `GameTest_AllOnes_ScoresTwenty`
9 - 게임의 모든 20번 투구가 1핀인 경우
10 - 예상 점수: 20점 (1 x 20)
11 - 이 테스트는 roll() 메서드가 값을 저장하고 score() 메서드가 합산하는 기본 구조를 검증합니다
12 - 거터 게임의 경우 이 테스트는 실패하므로 roll()이 실제로 값을 저장해야 함을 증명합니다
13
14 ## 구현 계획
15
16 ### 헤더 파일: `Game.h`
17 ```cpp
18 #pragma once
19
20 class Game {
21 public:
22 void roll(int pins);
23 int score();
24 }
```

# RED 2 – Test Code 변환 후 검토

마찬가지로 변환된 Test Code 가 Plan.md 의 목표에 맞도록 작성되었는지 확인한다.

The screenshot shows an IDE with two tabs: 'Plan.md' and 'GameTest.java'. The 'Plan.md' tab on the left contains the following text:

**Plan.md — 두 번째 TDD 사이클**

**기능**

모든 투구가 1핀인 게임 — 20번 모두 1핀, 총점 = 20

**근거**

거터 게임은 score()가 항상 0을 반환해도 통과합니다. 이 테스트는 roll()이 실제로 핀 수를 누적하고 있음을 강제합니다. 스페어·스트라이크 없이 순수 합산만 검증하는 가장 단순한 "정상 케이스"입니다.

**테스트 설계 (코드 작성 전 의도)**

```
Game game = new Game();
20번 roll(1) 호출
assertEquals(20, game.score());
```

**패키지**

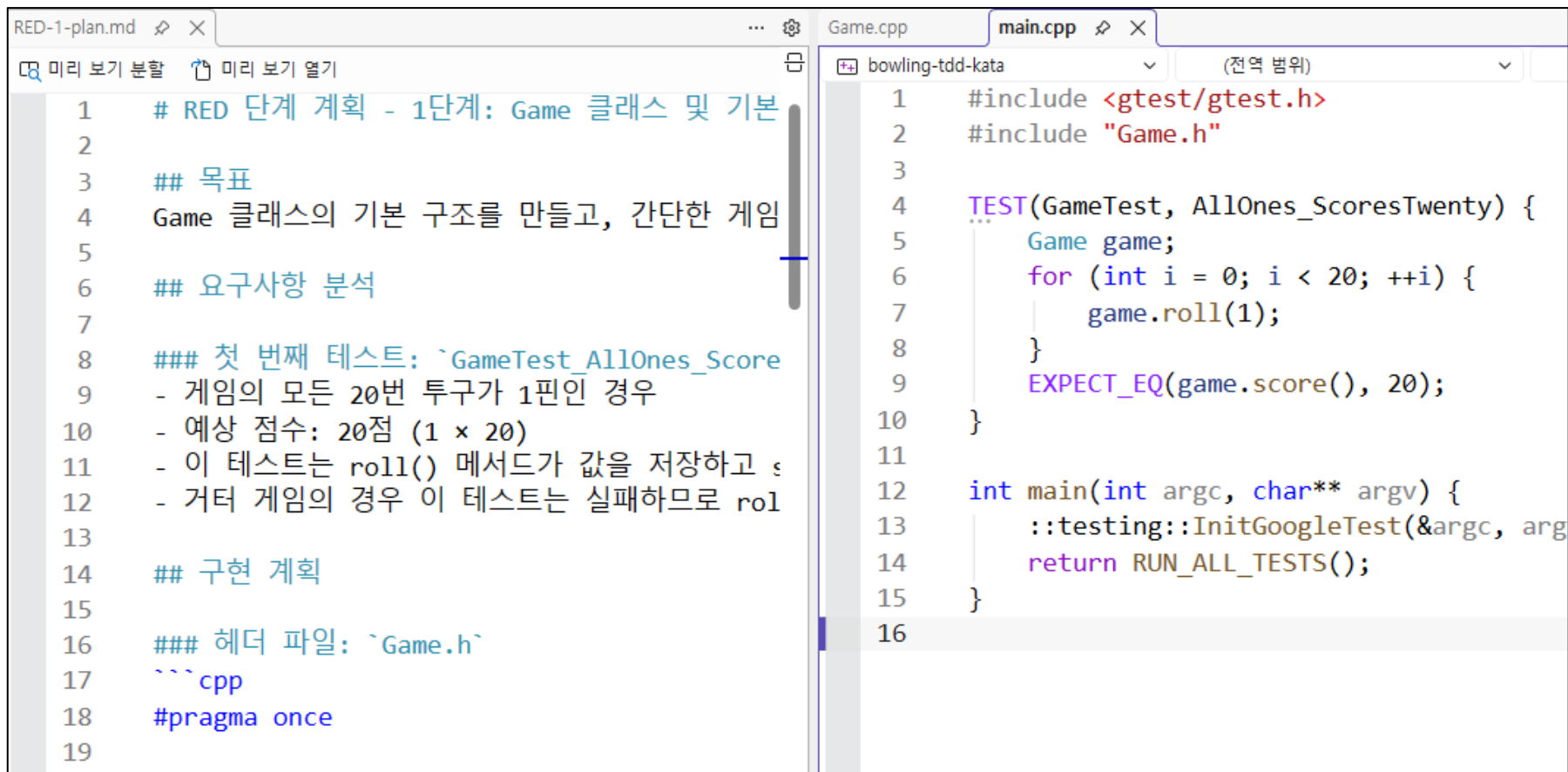
```
org.example.bowling
```

The 'GameTest.java' tab on the right contains the following code:

```
1 package org.example.bowling;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertEquals;
7
8 class GameTest {
9
10 @Test
11 @DisplayName("거터 게임은 총점이 0이다")
12 void gutterGameScoresZero() {
13 Game game = new Game();
14 for (int i = 0; i < 20; i++) {
15 game.roll(pins: 0);
16 }
17 assertEquals(expected: 0, game.score());
18 }
19
20 @Test
21 @DisplayName("모든 투구가 1핀이면 총점이 20이다")
22 void allOnesScoresTwenty() {
23 Game game = new Game();
24 for (int i = 0; i < 20; i++) {
25 game.roll(pins: 1);
26 }
27 assertEquals(expected: 20, game.score());
28 }
29 }
30
```

# RED 2 – 테스트 검토

테스트가 Plan 에 맞춰 개발 되었는지 확인한다.



The image shows a code editor with two panels. The left panel displays a file named 'RED-1-plan.md' containing a test plan. The right panel displays a file named 'main.cpp' containing the implementation of the test.

```
RED-1-plan.md
1 # RED 단계 계획 - 1단계: Game 클래스 및 기본
2
3 ## 목표
4 Game 클래스의 기본 구조를 만들고, 간단한 게임
5
6 ## 요구사항 분석
7
8 ### 첫 번째 테스트: `GameTest_AllOnes_Score`
9 - 게임의 모든 20번 투구가 1핀인 경우
10 - 예상 점수: 20점 (1 x 20)
11 - 이 테스트는 roll() 메서드가 값을 저장하고 s
12 - 거터 게임의 경우 이 테스트는 실패하므로 rol
13
14 ## 구현 계획
15
16 ### 헤더 파일: `Game.h`
17 ```cpp
18 #pragma once
19
```

```
main.cpp
1 #include <gtest/gtest.h>
2 #include "Game.h"
3
4 TEST(GameTest, AllOnes_ScoresTwenty) {
5 Game game;
6 for (int i = 0; i < 20; ++i) {
7 game.roll(1);
8 }
9 EXPECT_EQ(game.score(), 20);
10 }
11
12 int main(int argc, char** argv) {
13 ::testing::InitGoogleTest(&argc, arg
14 return RUN_ALL_TESTS();
15 }
16
```

## GREEN 2 – 빌드 및 테스트 통과를 확인한다.

### 빌드 및 테스트 통과를 확인

-  빌드 성공 - 테스트 실행
- PowerShell(.\\bowling-tdd-kata\\x64\\Debug\\bowling-tdd-kata.exe --g  
 L Note: Google Test filter = GameTest.AllOnes\_ScoresTwenty  
 [=====] Running 1 test from 1 test case.  
 [-----] Global test environment set-up.  
 [-----] 1 test from GameTest  
 [ RUN ] GameTest.AllOnes\_ScoresTwenty  
 [ OK ] GameTest.AllOnes\_ScoresTwenty (0 ms)  
 [-----] 1 test from GameTest (0 ms total)  
  
 [-----] Global test environment tear-down  
 [=====] 1 test from 1 test case ran. (0 ms total)  
 [ PASSED ] 1 test.



## REVIEW 2

생성된 코드에 대해 리뷰한다.

A screenshot of a code editor window. The title bar shows 'Game.cpp' and 'main.cpp'. The editor content shows a C++ file named 'Game.cpp' with the following code:

```
1 #include "Game.h"
2
3 void Game::roll(int pins) {
4 rolls[current_roll++] = pins;
5 }
6
7 int Game::score() {
8 int score = 0;
9 for (int i = 0; i < 20; ++i) {
10 score += rolls[i];
11 }
12 return score;
13 }
14
```

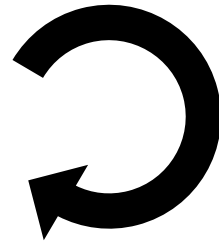
# 나머지 Cycle

나머지 Cycle 을 과제로 수행하도록 한다.

과제 수행 시 human-in-the-loop 가 잘 수행되었는지 체크용으로 Commit 을 만든다.



RED 단계에서 생성된  
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



## Chapter 04

# Agentic Engineering은 Over-engineering일까?

SW 품질을 위한 Agentic Engineering

## 아래 내용이 Agentic Engineering에 위배되는지 고민해보자. 1

- E2E 테스트 스크립트 적절히 추가해줘
- 이후 실행해보고 Fail되면 원인 분석 후 소스코드 or 스크립트를 개선해줘

## 아래 내용이 Agentic Engineering에 위배되는지 고민해보자. 2

위 방식은 완벽히 AE를 어기는 방식이다.

아래와 같이 해야 한다.

- E2E 테스트 스크립트 어떤 것을 만들면 좋을지 PLAN.md에 적어보라고 지시
  - 이후 인간의 검토 및 AI와 토론
- "테스트 스크립트 만들고 실행해" 지시
- (만약 Fail 발생시) 원인 분석과 수정 방향을 DEBUG.md에 작성해봐
  - 이후 인간의 검토 및 AI와 토론
- "수정 후 테스트까지 해봐" 지시

# Test Code 만들 때에도 매번 PLAN 문서부터 만들라고 시켜야 할까?

## TDD 할 때

- Code로 만들어진 것도 Plan 이라고 할 수 있지만, AI와 토론이 필요하다.
- Red 단계는 설계에 해당되며, 깊은 고민이 필요하다.
- Text로 만들어진 Plan을 작성, 검토 후에, Code로 만드는 것이 바람직하다.
- Text로 Plan 작성 → Code로 Plan 작성 (Unit Test)

## 가볍게 Test Case 1개 만들 때

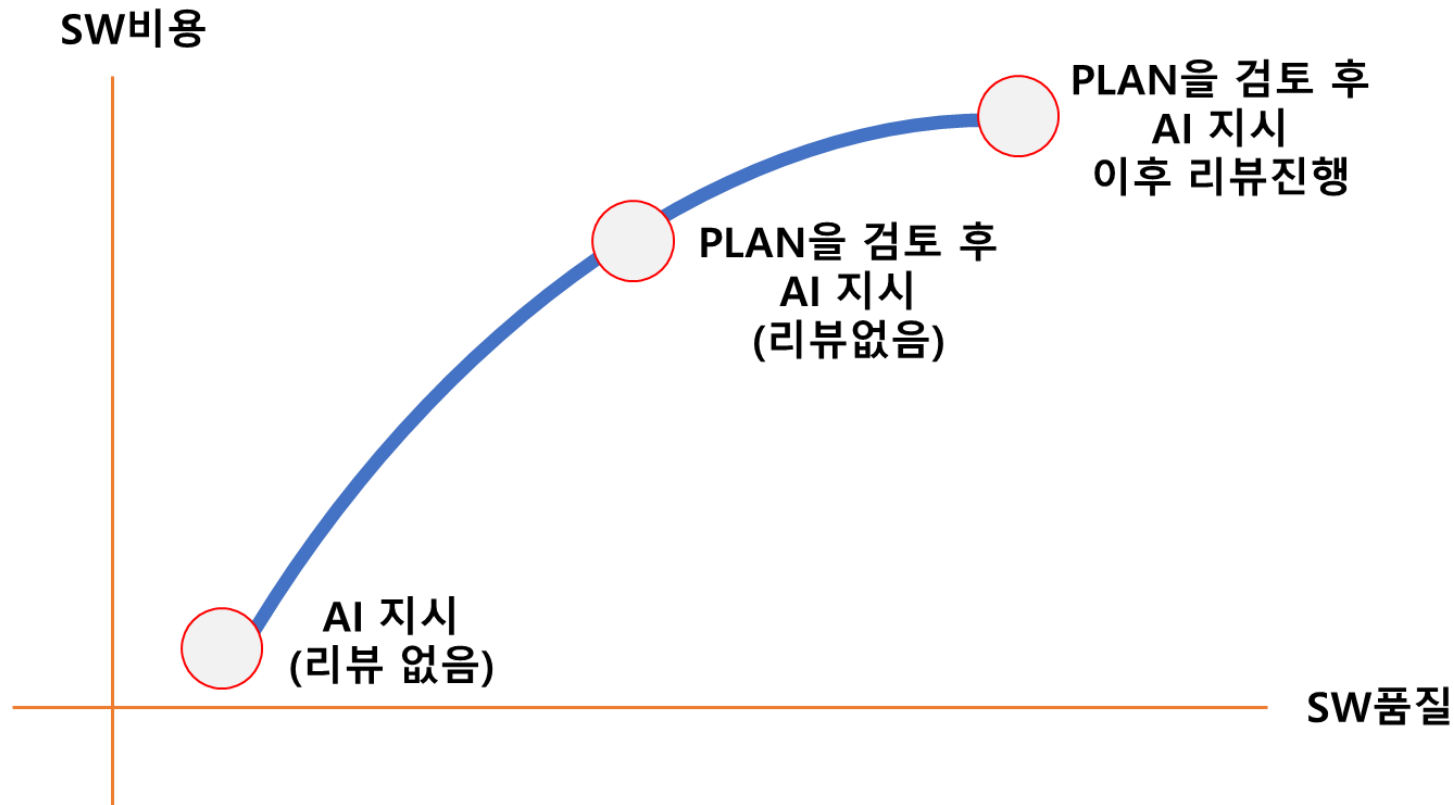
- [PLAN.md](#) 없이 대화로 어떤 것을 만들지 먼저 계획을 듣자. PLAN Mode만 사용해도 좋다.

## Test Case 여러 개 만들 때

- [PLAN.md](#) 를 만들고, 사람 리뷰 & AI와 토론을 먼저 한다.

# Agentic Engineering은 오버엔지니어링?

검토된 PLAN 없이 AI에게 작업을 시키고 리뷰를 하지 않는 것은 최악이라는 것은 동의할 수 있다.  
AI에게 시킨 후 리뷰만 하면 되지 않나? 굳이 PLAN 문서로 대화 후, 시키고, 리뷰까지 해야 할까?



# Agentic Engineering은 오버엔지니어링?

## 이유 1

- AI는 사람의 의도보다 더 많은 것을 구현한다. 따라서 AI가 무엇을 할지 미리 듣고 협의를 해야 한다.

## 이유 2

- AI에 맡기는 만큼 "요청문서 관리" 중요하다. "요청문서가 관리되지 않으면 안된다"

## 이유 3

- AI에게 시키고 후속 검증이 필요하다.



## 비유 - 억 단위 인테리어 어떻게 할까?

회사 입장에서는 SW 개발자가 만드는 제품은, SW 비용들이 포함된 고가의 제품이다.  
이 제품을 만들 때는 '인테리어 할 때' 처럼 신중해야 한다.

인테리어 업자에게 전화로  
"이쁘게 만들어 주세요"

VS

전체적으로 '이쁘게'에 대해  
요구사항을 정리하고  
각 단계별로 어떻게 만들 것인지 소통하고  
한 단계 공사 후, 결과물에 대해 검토하고  
수정 요청

## MVP, PoC, 연구 수준이라면?

하네스 엔지니어링으로 충분하다.

Agentic 엔지니어링은 오버엔지니어링이다.

**대신 실제 고객에게 제공되는 SW제품이라면?**

- Agentic Engineering은 오버엔지니어링이 아니다. 필요한 엔지니어링이다.

Chapter 04

# 지속적인 발전을 위한 Agentic Engineering

SW 품질을 위한 Agentic Engineering

## Agentic Engineering의 단점

- 똑딱 만드는 하네스 엔지니어링에 비해, 개발 속도가 매우 느리다.
- Self 검토 및 AI와 토론, 팀 동료와 토론 및 공유 시간이 많이 들기 때문이다.
- 클린 코드 일수록 이 시간을 줄일 수 있지만, 여전히 리뷰 시간은 길다.

하지만 SW 품질을 위해 반드시 필요한 투자 시간이다.

## SW품질 확보 외, Agentic Engineering의 장점

실력이 꾸준히 향상된다. AI로 인해 실력이 정체되는 것을 방지할 수 있다.

- 연차가 쌓일수록 SW 검토 실력이 지속적으로 늘어야 한다. 이건 중요한 문제이다.

반면, 하네스 엔지니어링에 과도하게 의존할 경우 코드 구현 역량이 점차 약화될 수 있다.

구현 경험이 줄어들수록 코드의 동작을 깊이 이해하는 능력과 검토, 판단 역량 역시 함께 약화될 가능성이 높다.

# 피할 수 없는 개발 인력의 축소, 어떻게 대응할까?

AI와 협업으로, 적은 인원으로도 SW 개발이 가능하다.

하네스 엔지니어링이 발전할수록 적은 실력으로도, OS급 SW개발도 가능해진다.

이런 발전 속도에서 SW 개발자의 경쟁력은 어디서 나올까?

- AI 없이도 개발할 수 있는 기본 SW 훈련 수준
  - SW 훈련이 잘 되어있을 수록 더 심도 있는 수준의 AI 토론이 가능하다.
- AI에게 사고를 의존하지 않고, AI 작업물에 대해 비판적 사고를 갖는다.
  - AI에게 냉철하게 질문한다.
- 알고리즘, 자료구조 능력 UP / 문제 분해 능력
  - AI가 만들 구현방식이 미리 예상이 되며, 이에 맞추어 Phase 분리를 잘 한다.
- AI가 만든 작업물을 검토하는 능력
  - 클린코드에 대한 고민을 많이 하여, 코드를 빠르게 이해하고 클린코드 방향성을 빠르게 캐치하는 능력

**Agentic Engineering은 개발 역량을 지속적으로 올릴 수 있는 엔지니어링이다.**

**감사합니다.**